

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 03 · LECTURE NOTES

Linear Programming

The most useful special case.

MODULE OUTCOMES

Put a problem into standard form.

Explain the geometry — polyhedra, vertices, and why the optimum is at one.

Explain the simplex method and its complexity paradox.

Model with linear programs, including the standard tricks.

Recognise when a problem is secretly linear.

1 The geometry

THE OPTIMUM IS AT A VERTEX

The feasible set of a linear program is a polyhedron — an intersection of finitely many half-spaces — and an intersection of convex sets is convex (Module 02 §2), so the feasible region is convex by construction.

A linear objective has a constant gradient, so there is no interior point where the gradient vanishes. Moving in the improving direction always improves, **until a constraint stops you**.

So an optimum, where one exists, is attained at a vertex of the polyhedron. (The optimal face may be an edge or a higher-dimensional face, in which case its vertices attain the optimum too, so looking only at vertices loses nothing.)

That reduces a continuous optimisation over infinitely many points to a finite search over vertices — which is the key structural fact. **There are exponentially many vertices**, so enumeration is not available and an intelligent walk is required (§2).

STANDARD FORM: minimise $c'x$ subject to $Ax = b, x \geq 0$

ANY linear program converts:

```

maximise  $c'x$           -> minimise  $-c'x$ 
 $a'x \leq b$              ->  $a'x + s = b, s \geq 0$  (slack)
 $a'x \geq b$              ->  $a'x - s = b, s \geq 0$  (surplus)
 $x$  unrestricted        ->  $x = x^+ - x^-$ , both  $\geq 0$ 
 $|x|$  in the objective ->  $x = x^+ - x^-$ , minimise  $x^+ + x^-$ 
                        -- ONLY when MINIMISING  $|x|$ 

```

THE LAST IS THE MOST USEFUL TRICK HERE. It is how L1 regression, L1 regularisation and robust fitting become linear programs -- and why the lasso (CSCE 633 M04) has an LP formulation.

IT ALSO SHOWS THE LIMIT: minimising $|x|$ is fine; MAXIMISING $|x|$ is not, because $|x|$ is convex and max of a convex function is not a concave problem. The DIRECTION decides legality.

That the absolute-value trick is directional is the instructive part — it is Module 02's convexity calculus reappearing as a hard modelling constraint rather than as theory. **Whenever a reformulation seems too convenient, check which direction it works in**, and the answer is usually that it works in the convex direction only.

2 The simplex method

SIMPLEX WALKS EDGES FROM VERTEX TO VERTEX

Start at a feasible vertex. Examine the adjacent vertices along the edges of the polyhedron; move to one that improves the objective; repeat until no neighbour improves.

Convexity guarantees that a vertex with no improving neighbour is globally optimal (Module 02 §4) — so a purely local stopping test constitutes a global certificate. That is the property doing all the work, and without it the walk would prove nothing.

Degeneracy can stall it. Several different bases can describe the same geometric vertex, so a pivot changes the algebra without moving — and in principle the algorithm can cycle forever between them. **Bland's rule guarantees termination** by imposing a deterministic tie-break, **at a real cost in speed**, so implementations use faster rules and fall back to Bland's only when stalling is detected.

And the pivoting rule is where the engineering is. Which improving neighbour to move to determines the path length, and **no pivoting rule is known to be polynomial** — a question open since the 1950s, and related to the Hirsch conjecture on polytope diameters, which was disproved in 2010.

EXPONENTIAL IN THEORY, EXCELLENT IN PRACTICE

Klee and Minty constructed a deformed hypercube on which the standard pivoting rule visits every one of its 2^n vertices. So the worst case is exponential, provably, for Dantzig's rule and for most others.

On real problems simplex takes a number of pivots roughly linear in the number of constraints — typically two to three times m . **The gap between worst case and typical behaviour is enormous** and was an embarrassment to complexity theory for decades.

Smoothed analysis explains it. Spielman and Teng showed that for *any* instance, a small random perturbation of its data makes simplex run in expected polynomial time — **so the bad instances are isolated and fragile**, destroyed by arbitrarily small noise, which is why they do not arise from measured data.

This is the clearest example in the whole program of worst-case complexity failing to describe practice (Module 01 §2), **and smoothed analysis was invented specifically to close this gap** — a case where the theory was extended to match a stubborn empirical fact rather than the other way round.

3 Modelling with linear programs

Problem	Formulation	Note
Diet and blending	Minimise cost subject to meeting nutritional or compositional requirements.	The original application , from the 1940s, and still the standard teaching example.
Transportation and assignment	Ship goods from supply nodes to demand nodes at minimum cost.	The optimum is automatically integral — see the callout.
Maximum flow	Flow conservation at each node, capacity on each edge.	A linear program whose optimum is integral (CSCE 629), which is why the combinatorial algorithms work.
Shortest path	The dual of a unit flow problem.	Also integral; the dual variables are the distance labels, which is Module 04's interpretation arriving concretely.
ℓ_1 regression	Split each residual into positive and negative parts and minimise their sum.	Robust to outliers in a way least squares is not (CSCE 633 Module 01 §3) — and it is an LP, not a least-squares problem.
Chebyshev (minimax) fitting	minimise t subject to $ r_i \leq t$ for every i .	One extra variable and $2m$ constraints turns a minimax problem into an LP — a trick worth remembering.
Two-player zero-sum games	Minimax over mixed strategies.	Von Neumann's theorem is LP duality , and the dual program is the opponent's problem (Module 04).

WHY FLOW AND MATCHING ARE EASY

Their constraint matrices are totally unimodular — every square submatrix has determinant 0, +1, or −1. Node-arc incidence matrices of directed graphs have this property, as do the constraint matrices of bipartite matching.

When a constraint matrix is totally unimodular and the right-hand side is integral, every vertex of the polyhedron has integer coordinates. This follows from Cramer's rule: the vertex coordinates are ratios of determinants, and the denominators are ± 1 .

So solving the linear programming relaxation returns an integer answer with no rounding and no branching — the integrality constraint is satisfied for free, and the NP-hardness of integer programming (Module 09) simply does not apply.

That is the precise reason maximum flow, bipartite matching, and shortest path are polynomial while the travelling salesman problem is not. It is a difference of matrix structure, not of effort — and recognising total unimodularity in a formulation is one of the most valuable skills in this subject, because it converts a hard problem into an easy one at a stroke.

4 In practice

- **Use a solver. HiGHS is free, open source, and excellent;** Gurobi and CPLEX are substantially faster on hard instances and have free academic licences. Writing your own is for understanding, not for use.
- **Solvers implement both simplex and interior point methods** (Module 07) and select between them per instance — **and only simplex supports a warm start**, which matters enormously when re-solving a slightly modified problem, as branch and bound does thousands of times (Module 09).
- **Presolve matters enormously.** Removing redundant constraints, fixing variables whose values are forced, and tightening bounds frequently halves a problem before a single iteration runs — and on badly-written models it does far more than that.
- **Numerical conditioning is a real and frequent source of failure.** Coefficients spanning many orders of magnitude produce genuinely wrong answers, infeasibility reports on feasible problems, and non-reproducible results — which is CSCE 620 Module 01's lesson in another domain.
- **So scale your model. Keeping all coefficients within a few orders of magnitude is the cheapest robustness improvement available**, and it is CSCE 633 Module 02's **feature scaling arriving as a numerical concern rather than an optimisation one** — the same fix, for a related reason.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 15.093 — linear programming lectures (free, OCW)	The reference for §1 through §3 , including the geometry and the simplex development.
Spielman & Teng — Smoothed Analysis of Algorithms (free)	The §2 resolution. Read the introduction for the argument even if the analysis is heavy.
Schrijver — Theory of Linear and Integer Programming	The reference for total unimodularity (§3) and the polyhedral theory. Library copy.
HiGHS, and the CVXPY linear programming examples (free)	A free solver that is genuinely competitive, with documentation on the §4 practical concerns.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Convert three linear programs to standard form**, including one with an absolute value and one with unrestricted variables.
- 2 **Show the absolute-value trick fails when maximising**, with a concrete instance.

- 3 Implement simplex for small problems and verify against a solver.
- 4 **Construct the Klee–Minty cube** for $n = 4$ and count the pivots your implementation takes.
- 5 **Measure pivot count against constraint count** on random instances and plot it.
- 6 Formulate and solve a transportation problem. **Verify the optimum is integral without being asked to be.**
- 7 **Formulate L_1 regression as an LP** and compare its fit against least squares on data with outliers.
- 8 Formulate Chebyshev fitting and compare against both.
- 9 **Verify total unimodularity** of a small node-arc incidence matrix by checking submatrix determinants.
- 10 **Scale a badly-conditioned model** and show the solver's behaviour changes.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a linear program's optimum at a vertex?
- 2 Convert a general LP to standard form, naming each transformation.
- 3 Why does the absolute-value trick only work in one direction?
- 4 Describe simplex and say what convexity contributes.
- 5 What is degeneracy and how is cycling prevented?
- 6 State the simplex complexity paradox and its resolution.
- 7 Name seven problems that are linear programs.
- 8 What is total unimodularity and what does it buy?
- 9 Give five practical concerns when using an LP solver.